



# *INDIA.RUNS*

Build what next India runs on

Team Name : DSInnovators2026

Team Leader Name : Dinesh Singh

Problem Statement : Intelligent Candidate Discovery & Ranking Challenge (Redrob AI Hackathon)

## Solution Overview

- A two-stage hybrid ranking pipeline for the 100K-candidate pool: a fast TF-IDF semantic screen narrows the field, then a rule-based structured scorer (skills, title, experience, location, career progression, behavioral signals, honeypot detection) re-ranks and explains the shortlist, fully offline on CPU.
- Unlike keyword-matching or embedding-only systems, it explicitly reasons about skill credibility (how long a skill was actually used, not just listed), penalizes job-hopping and title inflation, and down-weights profiles that are behaviorally inactive or internally inconsistent.

## JD Understanding & Candidate Evaluation

- Requirements parsed from the JD: role/title synonyms, the 5-9 year experience band, required vs. preferred skills (embeddings, retrieval, vector search, Python, evaluation frameworks), location preference, and the JD's explicit disqualifiers and 'do not want' list (title-chasers, framework-tutorial profiles, services-only careers, CV/speech-only backgrounds).
- Fit is evaluated beyond keywords: skill credibility is weighted by how long a skill was actually used, title match spans current and past roles, career progression flags job-hopping or title inflation, domain alignment separates NLP/IR from CV/robotics, and 23 behavioral signals act as a multiplier, so a keyword-stuffed but inactive or off-domain profile still scores low.

## Ranking Methodology

- Retrieval and scoring: Stage 1 runs a TF-IDF cosine similarity screen across all 100,000 candidates to shortlist; Stage 2 re-scores the shortlist with the full structured feature set, then sorts by score descending with candidate\_id as a deterministic tiebreaker.
- Models and heuristics: scikit-learn TF-IDF vectorization (with an optional dense sentence-embedding upgrade path), difflib-based fuzzy skill and title matching against curated synonym groups, and rule-based scorers for experience, location, career progression and honeypot consistency.
- Combining signals: a weighted composite of skill match, title match, experience fit, location fit, progression and stability, domain alignment and semantic similarity, multiplied by a behavioral-activity factor and reduced by job-hopper and honeypot penalties, normalized to a single 0-1 score.

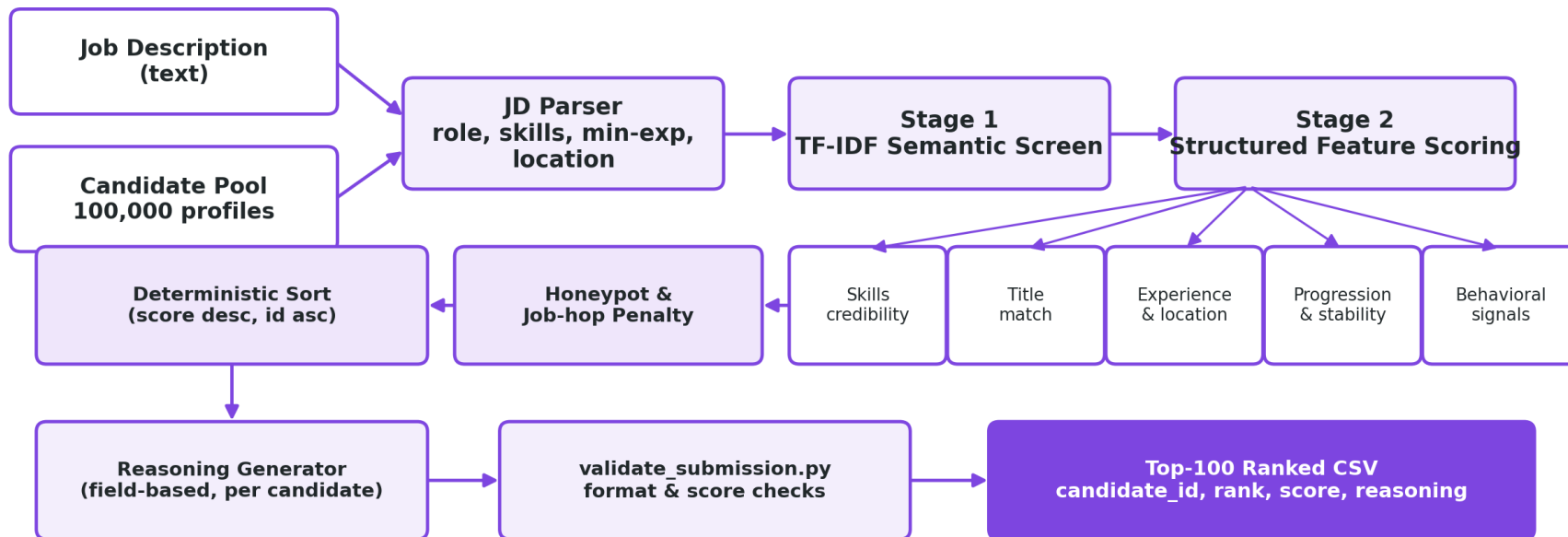
## Explainability & Data Validation

- Ranking decisions are explained with a 1-2 sentence reasoning string generated per candidate directly from that candidate's own computed score fields: title, years of experience, top matched skills, location, notice period and platform activity.
- Hallucination is prevented structurally: reasoning text is assembled from the score fields already computed for that candidate, never generated freeform, so nothing outside the candidate's own profile can appear in its reasoning.
- Inconsistent or suspicious profiles are caught by an 8-check honeypot scorer (zero-duration expert skills, career-history months vs. stated experience, impossible dates, out-of-range behavioral rates, seniority-vs-experience mismatch, keyword stuffing) with a weighted threshold, so borderline cases are down-weighted rather than harshly excluded.

## End-to-End Workflow

- Parse the job description into role, required/preferred skills, minimum experience and location.
- Stream and load all 100,000 candidate records from JSONL.
- Stage 1: TF-IDF semantic screen shortlists the strongest candidates.
- Stage 2: structured feature scoring plus honeypot consistency checks re-rank the shortlist.
- Deterministic sort, automatic reasoning generation, and a validated top-100 CSV as the final output.

## System Architecture



## Results & Performance

- All 100,000 candidates were ranked end-to-end in about 94 seconds on a single CPU core with roughly 3.2 GB peak memory: no GPU, no hosted LLM calls, and no network access during ranking. 0% honeypots landed in the final top-100, well under the 10% disqualification threshold.
- A qualitative check of the top-ranked candidates shows 6-9 year product-company ML/NLP engineers with retrieval and embeddings experience, Tier-1 education, recent platform activity and reasonable notice periods, matching the JD's own description of its ideal candidate rather than just keyword density.



## Technologies Used

- Python 3 and scikit-learn (TfidfVectorizer, cosine similarity) for the semantic-matching stage: lightweight and deterministic, with no GPU or network dependency, to satisfy the hackathon's compute constraints.
- difflib and curated skill/title synonym groups for fuzzy matching, avoiding a heavy NLP model just to normalize skill and title names.
- An optional sentence-transformers dense-embedding path that upgrades semantic matching when available, and automatically falls back to TF-IDF so the CPU-only, offline guarantee always holds.
- Streamlit for an interactive demo UI, and Python's standard csv, json and gzip libraries for all data input and output.

## Submission Assets

- GitHub repository: full source code, README with the exact reproduction command, requirements.txt and submission\_metadata.yaml.
- Sandbox / demo link: hosted small-sample runner as required by Section 10.5 of the submission spec.
- Final ranked submission CSV: top-100 candidates for the released job description, validated against validate\_submission.py.



# *INDIA.RUNS*

Build what next India runs on

# THANK YOU